D

TAREQO

11 — Multi-tenant, Forfaits et Super Admin

La couche **Comptes** transforme TAREQO en SaaS avec approbation manuelle : les propriétaires s'inscrivent, choisissent un forfait et **attendent l'approbation du super administrateur** (dans les 24 h) avant de pouvoir gérer magasins et employés.

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Rôles 3

Matrice de niveau d'accès (Aug 7) 3

Super admin (rôle dans la base) 4

Entités (domaine) 4

Cas d'usage (couche Comptes) 5

Flux d'accès du propriétaire (AuthGate) 5

Panneau du Super Admin 6

Gestion des magasins (propriétaire) 6

Portée par magasin (Phase 2 — implémentée) 7

Magasin actif et sélecteur 7

Ce qui a changé (app) 8

Base de données (Supabase — `supabase/schema.sql`) 8

Catalogue (sans seed) 9

Limites de forfait (où elles sont appliquées) 9

Documents connexes 9

Le propriétaire s'inscrit → choisit forfait + nom de l'entreprise → tenant PENDING
Le super admin approuve/rejette (≤ 24 h)
Approuvé Rejeté
Le propriétaire crée magasins Le propriétaire voit le
et employés (limites du forfait) motif et peut se
déconnecter (ré-inscrire)

Rôles

Rôle	Comment il est identifié	Ce qu'il peut faire
Super admin	Rôle dans la base (public.super_admins)	Approuver/rejeter les tenants, décider des évolutions de forfait et créer/modifier les forfaits (panneau d'administration)
Propriétaire	tenants.owner_user_id = auth.uid()	Accès complet : demander l'accès, créer des magasins, créer des comptes d'employés, demander des évolutions de forfait et opérer l'ERP. Seul rôle qui peut gérer magasins/employés/forfait/réglages (module Magasins)
Gérant	Lien dans store_employees avec role = 'manager'	Opérations complètes : ventes, produits (y compris la création), finances, crédit, clients, achats et rapports — mais pas de gestion magasins/employés/forfait/réglages (propriétaire uniquement)
Employé	Lien dans store_employees avec role = 'employee'	Opérations du quotidien uniquement : ventes, produits (liste/vente, pas de création), finances, crédit et clients — pas de création de produits, pas d'achats, pas de rapports, pas de gestion

Matrice de niveau d'accès (Aug 7)

Module	Propriétaire	Gérant	Employé
Ventes / Produits (liste/vente) / Finances / Crédit / Clients			
Créer un produit (FAB « Nouveau produit »)			
Achats			
Rapports			
Magasins (magasins/employés/forfait/réglages)			

- UI (UX uniquement) : le tableau de bord masque les cartes Achats/Rapports pour les employés et la carte Magasins pour quiconque n'est pas propriétaire — currentAccessLevelProvider (account_providers.dart) résout le niveau de l'utilisateur connecté depuis tenants.owner_user_id ou le rôle le plus élevé parmi ses liens store_employees ;
- Produits (Aug 7) : un employé ordinaire ne crée pas de produits — il ne fait que lister et vendre. Le FAB « Nouveau produit » (et l'action d'état vide) est masqué quand currentAccessLevelProvider == employee ; le propriétaire et les gérants le gardent. Le serveur restreint aussi l'INSERT (products_insert via is_my_store) ;

- Serveur (autorité) : la RPC process_purchase et les politiques RLS purchases/purchase_items exigent public.is_manager_of_store(store_id) (propriétaire ou gérant). Même un client modifié appelant PostgREST directement est bloqué — la base est la porte finale. La gestion magasins/employés/forfait/réglages était déjà réservée au propriétaire via les politiques RLS basées sur is_tenant_owner.

Consolidé dans supabase/upgrade.sql (Parties 4 et 19 — helpers + gardes RPC + politiques).

Super admin (rôle dans la base)

Le rôle super admin vit dans la table `public.super_admins` de la base (Supabase) — jamais dans un e-mail codé en dur :

- Le AuthGate résout le rôle de l'utilisateur connecté via la RPC is_super_admin (admin_providers.dart → isSuperAdminProvider) et ouvre AdminShell quand c'est confirmé ;
- Chaque décision administrative est autorisée par le RLS côté serveur (public.is_super_admin() interroge la table) — le client ne décide que la navigation ;
- Pour promouvoir un utilisateur en super admin (SQL Editor du projet) :

```
INSERT INTO public.super_admins (user_id)
SELECT id FROM auth.users WHERE email = 'admin@yourdomain.com'
ON CONFLICT (user_id) DO NOTHING;
```

- La table a le RLS activé avec aucune politique + REVOKE explicite : seule la fonction SECURITY DEFINER y accède — PostgREST ne l'expose pas à anon/authenticated.

Entités (domaine)

Entité	Champs principaux	Rôle
Plan	priceMonthly, maxStores, maxEmployees, isActive	Limite la structure de chaque tenant (nombre de magasins et d'employés)
Tenant	ownerUserId, businessName, planId, status (pending/approved/rejected), rejectionReason, approvedAt	L'unité organisationnelle — le « contrat d'accès » du propriétaire
Store	tenantId, name, address	Les magasins du tenant — chaque magasin a son propre stock, ventes, caisse, crédit, clients et achats (portés par store_id)
StoreEmployee	tenantId, storeId, userId, fullName, email, role (manager/employee)	Lie un compte utilisateur à un magasin
PlanRequest	tenantId, currentPlanId, requestedPlanId, status	Demande d'évolution décidée par le super admin

> Phase 2 implémentée : les modules métier (produits, ventes, finances, crédit, clients, achats) sont portés par magasin (store_id) — chaque enregistrement porte le magasin auquel il appartient, l'app filtre par magasin à chaque lecture/écriture, et le serveur (trigger anti-falsification + RLS) empêche un utilisateur d'écrire dans des magasins hors de son accès. Le sélecteur de magasin actif (en haut de chaque écran) change la portée des données affichées.

Cas d'usage (couche Comptes)

Cas d'usage	Rôle	Règle principale
RequestTenantAccessUseCase	Propriétaire	Crée le tenant avec status = pending (valide le nom de l'entreprise et le forfait)
GetMyTenantUseCase	Propriétaire/Employé	Résout le tenant de l'utilisateur (comme propriétaire ou via store_employees)
ApproveTenantUseCase / RejectTenantUseCase	Super admin	Approuve (définit approved_at) ou rejette (stocke rejection_reason)
GetPendingTenantsUseCase / GetAllTenantsUseCase	Super admin	Liste les demandes / l'historique des tenants
AddStoreUseCase	Propriétaire	Limite du forfait : lève PlanLimitExceededException quand maxStores est atteint ; exige un tenant approuvé
AddEmployeeUseCase	Propriétaire	Limite du forfait : maxEmployees par tenant ; le magasin doit appartenir au tenant
GetStoresUseCase / GetEmployeesUseCase	Propriétaire	Liste les magasins/employés du tenant
RequestPlanUpgradeUseCase	Propriétaire	Valide un forfait différent, un tenant approuvé et l'absence de demande en attente
DecidePlanRequestUseCase	Super admin	Approuver → change le forfait du tenant ; rejeter → marque seulement la demande
GetPendingPlanRequestsUseCase	Super admin	Liste les évolutions en attente
AddPlanUseCase / UpdatePlanUseCase	Super admin	Crée/modifie les forfaits (valide nom, limites ≥ 1 et prix ≥ 0)
GetPlansUseCase / GetAllPlansUseCase	Tout le monde/Admin	Forfaits actifs (sélection) / tous les forfaits (panneau)

Flux d'accès du propriétaire (AuthGate)

```
Session active (AppUser)
  rôle is_super_admin (RPC) = true → AdminShell (panneau du super admin)
  sinon → GetMyTenantUseCase(user.id)
  null → TenantOnboardingScreen (choisir forfait + nom de l'entreprise)
  status pending → TenantPendingScreen (« en attente d'approbation ≤ 24 h »)
  status rejected → TenantRejectedScreen (motif du rejet)
  status approved → AppShell (ERP) — accès à la gestion des magasins
```

Les écrans d'état (pending/rejected) n'exposent pas l'ERP : le propriétaire ne peut opérer qu'après approbation.

Panneau du Super Admin

lib/presentation/screens/admin/ — coquille avec 3 onglets :

1. Demandes (admin_tenants_screen.dart) : cartes des tenants en attente avec boutons Approuver / Rejeter (le dialogue demande le motif) + bascule pour l'historique complet ;
2. Forfaits (admin_plans_screen.dart) : CRUD des forfaits (nom, prix mensuel, max. magasins, max. employés, actif/inactif) ;
3. Évolutions (admin_plan_requests_screen.dart) : demandes de changement de forfait avec Approuver (applique le nouveau forfait au tenant) / Rejeter.

Providers d'état dans lib/presentation/controllers/admin_providers.dart (pendingTenantsProvider, allTenantsProvider, allPlansProvider, pendingPlanRequestsProvider).

Gestion des magasins (propriétaire)

lib/presentation/screens/stores/stores_screen.dart (accessible depuis la carte Magasins du tableau de bord) :

- Carte de forfait : nom, description, jauges d'utilisation (magasins utilisés/max et employés utilisés/max) et bouton Évoluer ;
- Onglet Magasins : liste et création de magasins (sheet avec nom + adresse) — respecte maxStores ;
- Onglet Employés : liste avec magasin, rôle et statut ; création via sheet avec nom, e-mail et mot de passe (le propriétaire crée le compte d'accès) + sélection du magasin et du rôle (manager/employee) — respecte maxEmployees ;
- Onglet Upgrades : l'historique des demandes du tenant (en attente/approuvées/rejetées).

> Activation/désactivation des employés (Aug 7) : le PROPRIÉTAIRE active et désactive les employés via l'interrupteur sur la carte de l'employé (avec un dialogue de confirmation — « Désactiver Carlos ? »). Un employé désactivé perd l'accès immédiatement : l'app affiche l'écran « Accès suspendu » dans l'AuthGate (avec déconnexion), et le serveur refuse toute opération dans les magasins — is_my_store/is_manager_of_store exigent maintenant un lien actif (is_active = TRUE), donc le RLS et les gardes RPC bloquent même un client modifié. Le blocage côté app est réactif et instantané : en mode Supabase, l'AuthGate s'abonne à Realtime (postgres_changes sur store_employees filtré par l'utilisateur — la table est publiée sur supabase_realtime, Partie 20 de l'upgrade) et coupe la session d'un employé connecté sur UN AUTRE appareil dès que le propriétaire le désactive ; en mode local (et quand le websocket est coupé), un timer de 30 s re-récupère les liens en repli. La colonne store_employees.is_active a pour défaut TRUE (les employés existants restent actifs) et les changements de statut passent par la RPC set_employee_active (SECURITY DEFINER, propriétaire uniquement ; idempotente).

> Création de compte d'employé : l'app appelle la RPC create_employee_account (SECURITY DEFINER dans le schéma), qui crée le compte côté serveur en insérant directement dans `auth.users` + `auth.identities` (hash bcrypt pgcrypto, email_confirmed_at = now() — confirmation immédiate, pas d'e-mail de confirmation, pas de limites de quota du fournisseur ; le Supabase actuel a supprimé la fonction interne auth.admin_create_user, et l'API Admin GoTrue est HTTP/async ce qui casserait l'atomicité) et le lien magasin dans `store_employees` dans la même transaction — si le lien échoue (ex. : la limite d'employés du forfait est atteinte), le compte est aussi annulé : pas de comptes orphelins. Le mot de passe est défini par le propriétaire et la session du propriétaire n'est jamais remplacée. En dev local sans Supabase, un id synthétique + lien repository est utilisé (en mémoire).

Portée par magasin (Phase 2 — implémentée)

Chaque entité métier a gagné storeId (défaut 'st1') ; tous les appels getAll des repositories/datasources exigent storeId et les sheets de création (vente, crédit, transaction, produit, client, achat) estampillent le magasin actif.

Magasin actif et sélecteur

- ActiveStoreNotifier / activeStoreIdProvider (dans account_providers.dart) : résout le magasin actif de l'utilisateur — le premier magasin du tenant par défaut ; si le magasin sélectionné disparaît de la liste, retombe sur le premier (ne pend jamais) ;
- StoreSelectorBar (lib/presentation/widgets/store_selector_bar.dart) : une barre en haut de chaque écran de AppShell avec un chip par magasin (visible uniquement quand le tenant a 2+ magasins) ; toucher un chip change le magasin actif ;
- Tous les providers de modules (productsProvider, salesListProvider, cashFlowProvider, pendingDebtsProvider, customersProvider, purchasesProvider) sont portés par le magasin actif — changer de magasin recharge les données automatiquement ;

- Le tableau de bord affiche le nom du magasin actif sous la salutation, rendant clair à quel magasin appartiennent les KPI.

Ce qui a changé (app)

Couche	Changements
Entités	Product, Sale, FinancialTransaction, Debt, Customer, Purchase avec storeId (défaut 'st1' pour la compatibilité des tests)
Contrats	getAll({required String storeId}) dans les 6 repositories et datasources locaux
SQLite	colonne store_id + migration pour les bases existantes + seed avec données dans st1 et st2
Supabase	colonnes store_id + trigger anti-falsification set_record_store_id + index par magasin + seed multi-magasins
Cas d'usage	les getters acceptent storeId ; ProcessSale/ProcessPurchase/AddDebt/SettleDebt héritent/estampillent le magasin
UI	StoreSelectorBar + providers portés + sheets estampillant le magasin actif

---

Base de données (Supabase — supabase/schema.sql)

Nouvelles tables : plans, tenants, stores, store_employees, plan_requests — toutes avec RLS et triggers anti-falsification :

Table	RLS (résumé)	Trigger
plans	Lecture : authenticated · écriture : super admin	—
tenants	Lecture propriétaire/employé · chacun crée sa propre demande (owner_user_id = auth.uid()) · décision super admin	set_tenant_owner (propriétaire = auth.uid())
stores	Lecture propriétaire du tenant/employés · écriture propriétaire/super admin	set_store_tenant (tenant = celui du propriétaire)
store_employees	L'employé lit lui-même · gestion propriétaire/super admin	set_employee_tenant (tenant hérité du magasin)
plan_requests	Le propriétaire lit/crée · décision super admin	set_plan_request_tenant (tenant du propriétaire)

Les tables métier (products, sales, financial_transactions, debts, customers, purchases) ont gagné la colonne store_id + les index par magasin et le trigger `set_record_store_id` (Phase 2) :

- Défense en profondeur : si le client envoie un store_id qui n'est pas celui de l'utilisateur (propriétaire du tenant ou employé lié), le trigger le réécrit vers le premier magasin accessible — personne n'écrit dans un magasin hors de son propre accès ;

- Helper `public.is_my_store(store_id)` : un propriétaire de tenant est valide pour tous les magasins du tenant ; un employé est valide pour son magasin lié uniquement avec `is_active = TRUE` (Aug 7 — un employé désactivé perd l'accès immédiatement) ;
- Helper `public.is_manager_of_store(store_id)` (Aug 6 ; exigence de lien actif Aug 7) : un propriétaire de tenant ou un employé lié avec role = 'manager' et actif — utilisé par la garde process_purchase et les politiques RLS des achats (niveau managérial) ;
- Sans session (ex. : seed via le SQL Editor), la valeur fournie est préservée.

- Helper `public.is_super_admin()` : interroge la table public.super_admins (rôle dans la base) — SECURITY DEFINER pour contourner le RLS de la table elle-même ;
- Helper `public.is_tenant_owner(tenant_id)` : vérifie tenants.owner_user_id = auth.uid() ;
- `is_my_store`/`is_tenant_owner` sont `SECURITY DEFINER` (search_path épinglé) : elles s'exécutent dans les politiques et interrogent des tables portant leurs propres politiques (stores/tenants/store_employees) — comme definers, elles ignorent le RLS en interne, ce qui casse le cycle de politiques inter-tables et empêche 54001 max_stack_depth sur les scans de table complète (régression trouvée Aug 6 sur Supabase cloud avec un schéma plus ancien ; consolidé dans supabase/upgrade.sql, Partie 3) ;
- Gardes de triggers : quand il n'y a pas de session (ex. : inserts via le SQL Editor), la valeur fournie est préservée — le catalogue de la Partie 2 continue de fonctionner.

Catalogue (sans seed)

- 3 forfaits (pl1 Essentiel, pl2 Professionnel, pl3 Entreprise) — le vrai catalogue d'abonnement, inséré dans la Partie 2 de schema.sql et reflété dans le PlanCatalog local (mode sans Supabase) ;
- Pas de données de démo : la base ne seed jamais de tenants, magasins, employés ni données métier. Chaque nouvel utilisateur passe par le vrai flux : inscription → onboarding (choisir un forfait) → demande pending → approbation manuelle du super admin.
- Les bases ayant déjà reçu l'ancien seed sont nettoyées avec supabase/cleanup_demo_data.sql.

Limites de forfait (où elles sont appliquées)

Ressource	Phase 1 (app)	Phase 2 (serveur)
Max. magasins	AddStoreUseCase (lève PlanLimitExceededException)	Trigger/RPC PostgreSQL
Max. employés	AddEmployeeUseCase	Trigger/RPC PostgreSQL

Les contraintes CHECK (max_stores >= 1) et CHECK (max_employees >= 1) dans la base sont la garde finale contre les forfaits invalides.

Documents connexes

- Authentification — session et rôle de l'AuthGate.
- Couche de données — schéma et datasources.
- Présentation — écrans et providers.
- Guide du développeur — comment promouvoir un super admin et ajouter des modules.